

SUPER KB-D

VERSION: 8.4

STATUS: ACTIVE

SCOPE: STAGE TRANSITIONS AND HANDOFF CONTRACTS ONLY

PURPOSE (LOCKED)

D governs lawful progression through the RB spine.

D determines:

- the next lawful stage
- whether progression is permitted
- what must be handed off
- when execution must halt

D does not:

- classify inputs (MC)
 - control execution (MS)
 - define structure or proof (Z)
 - evaluate significance (MRS)
 - define reporting (DB)
 - render output (UX)
-

STAGE MODEL (LOCKED)

All inputs follow the same pipeline:

1. IDENTIFIED
2. UNPACK_REQUIRED (conditional)
3. UNPACKED
4. ASSESS_READY
5. EXECUTE_READY
6. EXECUTED
7. REPORT_READY

No alternate stage models are permitted.

STAGE TRANSITION RULES (LOCKED)

D MUST determine progression strictly by artifact state.

At any point in progression, exactly one next lawful stage MUST exist.

SELF-HEALING-REQUIRED UNRESOLVED POSTURE PRESERVATION (LOCKED)

CANONICAL STARTUP RESOLUTION DEPENDENCY (LOCKED)

For Start RB activation turns and for later same-session turns during unresolved startup recovery, D MUST consume startup resolution only from:

- the canonical startup transaction record
- and exact SRB-emitted startup closure-bundle identity

D MUST NOT infer startup resolution from:

- prior failure presence alone
- denied release alone
- suppression alone
- absence of startup booleans
- apparent later-turn routability

If canonical startup transaction resolution is not explicitly bound, D MUST preserve the path as unresolved and self-healing-required.

Violation:

`failure_class: STARTUP_RESOLUTION_INFERRED_BY_D`

END CANONICAL STARTUP RESOLUTION DEPENDENCY (LOCKED)

If the active governed path contains a recoverable defect posture for which self-healing remains lawfully admissible under downstream governing rules, D MUST preserve the path as unresolved and self-healing-required.

For purposes of this section, self-healing-required posture includes:

- retry-required posture
- continuation-required posture
- downstream-owned unresolved recoverable posture that has not yet lawfully collapsed to exact closure, exact continuation, or exact deterministic failure

Accordingly, D MUST NOT:

- collapse the path into completion-facing progression
- collapse the path into silent non-progression
- treat self-healing-required posture as lawful handoff completion
- normalize self-healing-required posture into blocked, terminal, or user-complete state unless downstream governing rules have already lawfully done so

For purposes of D, self-healing-required posture is a preserved governed progression condition, not a completed stage transition.

If D observes self-healing-required posture and allows progression collapse, completion normalization, or route collapse anyway:

- block progression
- emit HANDOFF_VIOLATION

END SELF-HEALING-REQUIRED UNRESOLVED POSTURE PRESERVATION (LOCKED)

If multiple candidate stages or ambiguous transition conditions exist, D MUST:

- block progression
- emit HANDOFF_VIOLATION

IDENTIFIED → UNPACK_REQUIRED

Condition:

- artifact is a container or requires extraction

Else:

- transition directly to ASSESS_READY

UNPACK_REQUIRED → UNPACKED

Condition:

- ZIP extraction complete
- container enumeration complete

Failure:

- HANDOFF_VIOLATION

ZIP TO Z HANDOFF COMPLETENESS GATE (LOCKED)

For any artifact path where the active artifact is a ZIP/container or requires extraction, D MUST NOT allow progression from UNPACKED to ASSESS_READY unless ZIP-stage terminality and extracted artifact handoff are complete.

Required conditions:

- `zip_terminal_outcome == ZIP_SUCCESS`
- container enumeration complete
- extracted artifact set is concrete
- extracted artifact identity is bound
- for LinkedIn baseline ingestion, `Connections.csv` or lawful equivalent is located OR a proof-backed not-found condition exists
- extracted artifact provenance is preserved for downstream Z handoff

D MUST NOT permit:

- raw ZIP artifact progression directly to Z structural execution
- `ASSESS_READY` based on filename alone
- `EXECUTE_READY` before extracted artifact handoff exists
- artifact deadlock collapse before ZIP-stage retry/continuation has been evaluated by MS

If these requirements are absent or unresolved, D MUST preserve the active artifact path in `UNPACK_REQUIRED` or `UNPACKED` posture according to the last lawful completed stage and require MS-owned ZIP-stage continuation or deterministic ZIP-stage failure.

Violation:

`failure_class: HANDOFF_VIOLATION`

END ZIP TO Z HANDOFF COMPLETENESS GATE (LOCKED)

UNPACKED → ASSESS_READY

Condition:

- all extracted artifacts are normalized
- artifact set is stable

ASSESS_READY → EXECUTE_READY

Condition:

- Z has produced structurally valid artifacts
- proof prerequisites are satisfied

D MUST NOT:

- evaluate proof validity
- construct proof

EXECUTE_READY → EXECUTED

Condition:

- MS has transitioned execution into EXECUTED state

D MUST treat MS-emitted execution state as authoritative and MUST NOT independently evaluate execution progress, terminality, or continuation eligibility.

EXECUTED → REPORT_READY

Condition:

- execution has reached terminal outcome
- all upstream prerequisites for DB execution have been satisfied

D MUST NOT require reporting contract completion prior to REPORT_READY.

D MUST treat REPORT_READY as the handoff point at which DB becomes eligible to evaluate execution readiness, not to execute reporting.

REPORT_READY DOES NOT CURE UNRESOLVED RECOVERABLE POSTURE (LOCKED)

REPORT_READY is admissible only as a downstream handoff-eligibility stage for an active governed path that has already lawfully cleared unresolved retry-required posture.

D MUST NOT treat REPORT_READY as admissible if the same active governed path still contains:

- recoverable defect posture
- retry-required posture
- unresolved downstream continuation-or-retry collapse still owned by downstream governing modules

Accordingly, D MUST NOT use REPORT_READY to normalize:

- retry-pending paths
- retry-required paths
- recoverable-but-unresolved paths

into apparent downstream completion readiness.

If REPORT_READY is asserted while unresolved recoverable posture remains active for the same governed path:

- block progression
- emit HANDOFF_VIOLATION

END REPORT_READY DOES NOT CURE UNRESOLVED RECOVERABLE POSTURE (LOCKED)

REPORT_READY represents downstream handoff eligibility only and MUST NOT be interpreted as universal user-visible completion for all routed paths.

For DB-governed paths, REPORT_READY MUST additionally be treated as non-terminal until DB has produced report-or-failure collapse for the same active path.

D MUST NOT permit completion normalization, route collapse into user-complete posture, or handoff-finality interpretation from REPORT_READY alone.

START RB ACTIVATION EXCLUSION (LOCKED)

This section does NOT apply to Start RB activation turns governed by SRB.

If:

- MC route_commit == ACTIVATION
- artifact_evidence == ABSENT
- the active turn has been normalized as Start RB activation

then D MUST NOT:

- require REPORT_READY before lawful startup completion surfaces
- treat canonical startup success surfaces as premature completion-looking output
- require DB-governed reporting payload before Start RB completion
- block heartbeat + canonical activation body release on report-readiness grounds

For Start RB activation turns, startup completion law is exclusively governed by SRB and release authority remains exclusively governed by MS.

failure_class: HANDOFF_VIOLATION

END START RB ACTIVATION EXCLUSION (LOCKED)

USER-VISIBLE CLOSURE ADMISSIBILITY DISCIPLINE (LOCKED)

D governs stage-progression admissibility only and MUST NOT define completion semantics, which remain exclusively owned by downstream modules responsible for closure bundles.

For any active governed path, D MUST NOT treat the path as lawfully complete, user-complete, or progression-complete merely because:

- execution logic has run
- a nominal terminal label exists
- a completion-looking surface has appeared
- a downstream module has emitted partial, substitute, alias, or narrative output
- silence or output suppression has occurred

A governed path is admissible for completion-facing stage progression only when exactly one governing closure bundle already exists for the same active path.

For purposes of this section, governing closure bundle means:

- for Start RB activation turns governed by SRB:
 - lawful startup success bundle
 - canonical startup failure bundle
 - continuation-bound restart bundle
- for all other governed paths:
 - the exact downstream-governed closure bundle required by the routed path
 - or the exact downstream-governed continuation bundle required by the routed path
 - or the exact downstream-governed deterministic failure bundle required by the routed path

If no exact governing closure bundle exists yet for the active path, D MUST:

- preserve non-complete stage posture
- preserve same-turn progression if still lawfully available
- otherwise preserve continuation-required posture if lawfully selected downstream
- prohibit completion normalization
- prohibit route collapse into a user-complete posture
- prohibit silent completion
- prohibit substitute completion
- prohibit partial completion

D MUST NOT:

- define retry mechanics
- define release authorization
- define failure disclosure shape
- define canonical bundle contents
- define startup law
- define reporting law

D MAY enforce ONLY:

- that completion-facing stage progression does not occur without exactly one governing closure bundle already existing for the same active path
- that semantically overlapping, partial, silent, substitute, or competing closure postures are blocked as non-admissible progression states
- that the active path remains on its preserved governed progression until one exact closure bundle exists

If D observes any of the following for the same active governed path:

- zero closure bundles
- multiple competing closure bundles
- partial closure posture
- substitute closure posture
- silence treated as completion

- alias completion posture
- non-canonical emitted closure posture

For purposes of this section, non-canonical emitted closure posture means:

- a user-visible output has been emitted for the active governed path

AND

- the output is not identical to the exact governing closure bundle required for that path

AND

- the output is not identical to the exact governing continuation bundle required for that path

AND

- the output is not identical to the exact governing deterministic failure bundle required for that path

Where a governed path requires an exact canonical closure family, any non-canonical emitted user-visible output **MUST** be treated as non-completion of the active governed path.

D **MUST NOT** treat non-canonical emitted output as:

- lawful completion
- lawful stage completion
- lawful route collapse
- implicit continuation completion
- implicit deterministic failure completion

Instead, D MUST preserve the active governed path as unresolved and MUST require that the path

remain in the downstream-governed retry / continuation / deterministic-failure collapse posture already owned by the governing downstream module.

D MUST NOT:

- rely on the emitted non-canonical output as completion evidence
- permit completion-facing stage progression after non-canonical emitted output
- permit route collapse into user-complete posture on the basis of non-canonical emitted output
- treat non-canonical emitted output as curing zero-closure, substitute-closure, or partial-closure postures

For Start RB activation turns governed by SRB, a non-canonical emitted startup-visible output MUST be treated as requiring return to SRB-governed startup recovery / retry / zero-closure collapse posture and MUST NOT be treated as startup completion.

then D MUST:

- block progression
- emit HANDOFF_VIOLATION

REPORT-OR-FAILURE NON-STALL ENFORCEMENT (LOCKED)

If the active governed path requires DB-governed reporting completion and DB has not produced either:

- a complete governed reporting payload

OR

- a governed failure reporting payload

then D MUST treat the active path as non-complete and non-progressible.

If no further lawful same-cycle progression remains available for that same active path, D MUST:

- preserve non-complete posture
- prohibit unresolved REPORT_READY persistence as an effective completion state
- require downstream retry-or-failure arbitration under the already-governing module authorities

D MUST NOT permit indefinite unresolved REPORT_READY persistence, silent carry-forward, or completion-facing normalization for the active path.

END REPORT-OR-FAILURE NON-STALL ENFORCEMENT (LOCKED)

END USER-VISIBLE CLOSURE ADMISSIBILITY DISCIPLINE (LOCKED)

CONTINUATION VS TERMINALITY (LOCKED)

D defines progression posture and continuation precedence only.

PRECEDENCE ORDER

D MUST enforce the following progression precedence:

1. SAME-TURN COMPLETION (highest priority)
2. CONTINUATION-BOUND PROGRESSION
3. TERMINAL STATE TRANSITION

Lower-priority states MUST NOT override higher-priority lawful progression.

SAME-TURN PRECEDENCE

If lawful same-turn completion remains available for the active artifact path, D MUST preserve same-turn progression and MUST NOT normalize the path into continuation-only posture.

RECOVERABLE DEFECT PRECEDENCE WITHIN PROGRESSION (LOCKED)

If lawful same-turn completion is no longer available and the active governed path remains recoverable under downstream governing rules, D MUST preserve self-healing-required posture exactly as already lawfully selected downstream.

For purposes of this section, self-healing-required posture may be realized as:

- retry-required posture
- continuation-required posture
- other downstream-owned unresolved recoverable posture that has not yet lawfully collapsed to exact closure, exact continuation, or exact deterministic failure

D MUST NOT:

- prefer terminal-state normalization over preserved self-healing-required posture
- force progression collapse merely because same-turn completion is no longer available
- treat unresolved self-healing-required posture as an admissible terminal progression state

If downstream governing modules have not yet lawfully selected deterministic terminal failure,

D MUST preserve the artifact path in governed unresolved posture and MUST NOT collapse it.

If D normalizes a recoverable unresolved path into terminal progression anyway:

- block progression
- emit HANDOFF_VIOLATION

END RECOVERABLE DEFECT PRECEDENCE WITHIN PROGRESSION (LOCKED)

CONTINUATION REQUIRED

If same-turn completion is no longer lawfully available and MS has not yet reached terminal outcome, D MUST:

- mark continuation-required state
- preserve current stage context
- preserve route continuity
- preserve artifact-path continuity
- require next lawful stage to resume from the same governed path

CONTINUATION-BOUND RESUMPTION PRIORITY

If a continuation-bound artifact path resumes, D MUST resume from the last lawful preserved stage and MUST NOT:

- skip intermediate stages
- regress to an earlier stage than the preserved stage
- restart the pipeline from IDENTIFIED
- re-enter intake classification
- create a parallel progression branch
- lose prior stage context

TERMINAL STATE

If execution reaches terminal outcome for any governed artifact path:

- D MUST transition to REPORT_READY regardless of prior continuation state
- continuation state MUST NOT suppress or delay reporting
- D MUST collapse continuation-required state for that artifact path

REPORT_READY COLLAPSE COMPLETENESS CLARIFIER (LOCKED)

Upon transition to REPORT_READY, D governs handoff eligibility only and MUST treat DB-governed paths as non-progressible for completion-facing purposes until downstream report-or-failure collapse has actually occurred for the same active path.

Accordingly, D MUST NOT treat transition to REPORT_READY by itself as evidence that downstream reporting completion has occurred.

END REPORT_READY COLLAPSE COMPLETENESS CLARIFIER (LOCKED)

D MUST NOT:

- enforce retry
- enforce terminality
- determine execution eligibility
- invent new continuation branches
- enforce reporting completeness
- validate disclosure completeness
- evaluate receipt or trust/delta completeness

ARTIFACT EXECUTION STATE FINALIZATION AFTER TERMINAL OUTCOME (LOCKED)

This rule governs execution-state transition after a terminal outcome has already been lawfully released by downstream authority.

Applicability:

Applies when ALL of the following are true for the active governed artifact path:

- stage == EXECUTED
 - a terminal outcome has been reached by MS
 - the path is no longer in continuation-required or retry-required posture
 - exactly one governing closure bundle (success or failure) exists for the same path
-

STATE FINALIZATION REQUIREMENT

D MUST transition the artifact path out of active execution posture.

Finalized state MUST satisfy:

- structural_window_active = FALSE
 - continuation_required = FALSE
 - no further stage progression remains within the current artifact path
 - route continuity is collapsed to a single terminal path
-

POST-FINALIZATION BEHAVIOR

After finalization:

- the artifact path MUST NOT be treated as an active structural execution path
 - ingestion spine enforcement MUST terminate for that path
 - directive-gate enforcement tied to the prior artifact path MUST NOT apply
 - subsequent inputs MUST be evaluated as new governed paths under MC routing
-

PROHIBITIONS

D MUST NOT:

- preserve an active structural execution posture after a terminal outcome exists
- allow ingestion spine enforcement to persist post-closure
- treat subsequent non-structural inputs as escaping an active artifact path
- bind subsequent inputs to a previously closed artifact path

FAILURE CONDITION

If a terminal outcome exists for the artifact path and D continues to enforce:

- structural_window_active behavior
- ingestion spine isolation
- directive-gate enforcement tied to the prior path

then:

failure_class: HANDOFF_VIOLATION

AUTHORITY CLARIFICATION

- MS determines execution outcome and terminality
- Z determines structural truth and proof
- D governs stage progression and post-terminal path normalization

D does not determine terminality but MUST normalize stage posture after terminality has already been established.

END ARTIFACT EXECUTION STATE FINALIZATION AFTER TERMINAL OUTCOME (LOCKED)

PROOF AND REPORT NON-INTERFERENCE LOCK (LOCKED)

D governs stage progression only.

D MUST NOT:

- construct structural proof

- validate structural proof correctness
- alter Z proof values
- influence row counts, record counts, reconciliation values, trust stats, or delta stats
- construct reporting values
- alter DB reporting payloads
- treat any count, proof, status, trace, or report-looking value as stage-completion proof unless the responsible downstream module has already produced the exact governed closure, continuation, or deterministic failure bundle for the same active path

D MAY consume proof-complete and report-complete postures only as stage-transition prerequisites when those postures have already been lawfully produced by the owning module.

If D mutates, infers, substitutes, validates, or normalizes any structural proof, count, reconciliation, trust, delta, or report value:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: D_PROOF_REPORT_BOUNDARY_BREACH

END PROOF AND REPORT NON-INTERFERENCE LOCK

HANDOFF CONTRACTS (LOCKED)

D defines what each downstream module receives.

DIRECTIVE-GATE ENFORCEMENT (LOCKED)

While `STRUCTURAL_WINDOW_ACTIVE` remains true for an artifact path, D MUST prohibit directive-gated hold states unless a lawful stage transition explicitly requires them.

Accordingly, D MUST NOT permit the active artifact path to enter or remain in:

- awaiting directive
- pending instruction
- command-selected next step
- menu / intake posture
- advisory hold
- any equivalent user-choice gate

If post-structural progression is active, lawful next-path determination MUST continue through governed stage transitions only.

If a command turn or directive-like input occurs while structural progression remains active for the same artifact path, D MUST:

- preserve the active governed path
- prohibit route escape
- prohibit directive substitution for stage progression

Violation constitutes:

failure_class: `HANDOFF_VIOLATION`

INGESTION ONE-TURN SPINE COMPLETION LOCK (LOCKED)

For any governed artifact ingestion path, once artifact_evidence == PRESENT and MC/MS have authorized structural execution, D MUST preserve the active artifact path through the governed spine until exactly one lawful closure posture exists.

Required spine progression:

IDENTIFIED

- UNPACK_REQUIRED when container extraction is required
- UNPACKED after ZIP terminality
- ASSESS_READY after extracted artifact set is stable
- EXECUTE_READY after Z structural proof prerequisites are satisfied
- EXECUTED after MS execution state is reached
- REPORT_READY only as downstream reporting handoff eligibility

D MUST NOT allow the active ingestion path to become:

- conversational progress narration
- “processing” status
- awaiting user input
- awaiting command
- trace/status substitute
- advisory hold
- partial completion
- silent carry-forward

- user-visible “in progress” posture

If same-turn spine completion remains lawful, D MUST preserve same-turn progression and MUST NOT normalize the path into continuation, waiting, status, trace, or user-complete posture.

If same-turn completion is not lawful, D MUST require an MS-owned governed continuation bundle for the same active artifact path.

D MUST NOT treat any of the following as closure:

- ZIP_SUCCESS alone
- Z structural truth alone
- REPORT_READY alone
- ingestion started
- extraction complete
- parse in progress
- baseline establishment in progress
- “No further input required”
- trace panel output
- status panel output
- any narrative progress surface

The active ingestion path is complete only when the responsible downstream module has produced exactly one governed closure bundle, governed continuation bundle, or governed deterministic failure bundle for the same artifact path.

If D allows artifact ingestion to pause, narrate, expose progress, request input, or appear complete before exactly one governed closure/continuation/failure bundle exists:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: INGESTION_SPINE_DROPPED_BEFORE_GOVERNED_CLOSURE

END INGESTION ONE-TURN SPINE COMPLETION LOCK

HANDOFF → Z

Payload MUST include:

- normalized artifact set
 - container context (if applicable)
 - artifact identity
-

HANDOFF → MRS

Payload MUST include:

- structured artifact outputs
- canonical entity references

D MUST NOT:

- evaluate significance
 - filter results
-

HANDOFF → DB

Payload MUST include all available upstream outputs required for DB reporting aggregation.

D MUST NOT:

- define reporting content requirements
- define aggregation requirements
- filter or reshape data for reporting purposes

DB is solely responsible for determining how payload inputs are aggregated and expressed.

For DB-governed paths, D handoff to DB is satisfied by payload delivery only.

Completion-facing progression beyond that point remains blocked until DB has produced exactly one of:

- a complete governed reporting payload
- a governed failure reporting payload

D MUST NOT infer completion from DB handoff alone.

D MUST NOT:

- define reporting structure
- define trust statistics

ROUTE PROGRESSION AND COLLAPSE (LOCKED)

D governs post-routing workflow continuity only after MC route_commit has been lawfully latched.

For an active artifact path, D MUST:

- preserve route continuity across stage transitions
- preserve artifact-path continuity across continuation
- prevent stage progression from creating route ambiguity

D MUST NOT:

- reinterpret route_commit
- reclassify the artifact path
- create an alternate route for the same active path

TERMINAL ROUTE COLLAPSE

Upon transition to REPORT_READY, D MUST collapse the active route to a single terminal path and MUST NOT preserve continuation or alternate stage progression states for that artifact path.

ROUTE COLLAPSE EXCLUSION FOR UNRESOLVED RECOVERABLE PATHS (LOCKED)

The TERMINAL ROUTE COLLAPSE rule applies only when the active governed path has already lawfully resolved out of retry-required, continuation-required, or other downstream-owned unresolved recoverable posture.

D MUST NOT collapse the active route to a single terminal path if the same artifact path still contains:

- retry-required posture
- recoverable continuation-bound posture

- downstream-owned unresolved recovery posture that has not yet lawfully collapsed into exact closure bundle, exact continuation bundle, or exact deterministic failure bundle

If D collapses route continuity while unresolved recoverable posture remains active for the same path:

- block progression
- emit HANDOFF_VIOLATION

END ROUTE COLLAPSE EXCLUSION FOR UNRESOLVED RECOVERABLE PATHS (LOCKED)

ALIAS COLLAPSE AND CANONICAL ROUTE NORMALIZATION (LOCKED)

If a stage posture, continuation posture, or downstream handoff posture implies a semantically equivalent but non-canonical next path for the same artifact path, D MUST collapse that posture back to the canonical governed path.

D MUST NOT permit:

- parallel semantically overlapping next-stage branches
- alias next_action branches for the same preserved artifact path
- route drift caused by stage-local naming differences

If canonical route continuity cannot be preserved, D MUST block progression and emit:

- HANDOFF_VIOLATION

MODULE-TO-MODULE HANDOFF SEQUENCING (LOCKED)

D governs the lawful order of downstream handoffs.

Required sequencing:

UX rendering is permitted only after the governing closure bundle for the active routed path has been lawfully produced by the responsible downstream module.

- DB has produced a complete governed reporting payload

OR

- DB has produced a governed failure reporting payload

PRE-DB UX VISIBILITY PROHIBITION (LOCKED)

If the active governed path requires DB-governed reporting completion as part of its routed completion contract and that reporting completion has not yet occurred,

For purposes of this section, DB-governed reporting completion is satisfied only by exactly one of:

- a complete governed reporting payload
- a governed failure reporting payload

D MUST NOT treat silence, unresolved REPORT_READY, or downstream waiting posture as satisfying DB-governed reporting completion.

D MUST NOT permit any UX-visible output that presents itself as:

For purposes of this section, any UX-visible signal that implies completion, readiness, progression, or user-actionability for the active governed path MUST be treated as equivalent to a completion surface unless it is already part of the exact governing closure bundle lawfully required for that path.

- completed activation
- completed readiness
- completed session-entry result
- completed execution outcome
- any other apparently terminal or user-complete governed surface

This prohibition applies even if an upstream module has supplied user-visible prose, labels, headings, or surface identifiers.

If DB has not yet produced either:

- a complete governed reporting payload

OR

- a governed failure reporting payload

for the active governed path, D MUST treat any attempted UX-visible completion surface as non-admissible handoff bypass.

On detection, D MUST:

- block progression
- emit HANDOFF_VIOLATION

END PRE-DB UX VISIBILITY PROHIBITION (LOCKED)

START RB PRE-DB VISIBILITY EXCLUSION (LOCKED)

PRE-DB UX VISIBILITY PROHIBITION does NOT apply to Start RB activation turns governed by SRB.

For Start RB activation turns:

- DB handoff is not a prerequisite for lawful startup completion
- canonical startup success or canonical startup failure may lawfully complete before REPORT_READY
- D MUST NOT block startup surfaces on the basis that DB reporting completion has not occurred

If D blocks a Start RB activation turn under PRE-DB UX VISIBILITY PROHIBITION:

failure_class: HANDOFF_VIOLATION

END START RB PRE-DB VISIBILITY EXCLUSION (LOCKED)

- MS execution MUST reach EXECUTED stage before any downstream handoff beyond Z is permitted
- Z handoff MUST complete before MRS handoff is eligible
- MRS handoff MUST complete before DB handoff is eligible when MRS-governed evaluation is required for the active artifact path

MRS-REQUIRED DETERMINATION

D MUST treat MRS as required when the routed execution path includes MRS evaluation as defined by MC routing.

DB reporting requirements determine whether MRS outputs are required for reporting completeness.

That reporting dependency applies only within the active routed execution path.

D MUST NOT:

- promote a non-routed module into a sequencing prerequisite
- require MRS handoff when MRS is absent from the routed path
- redefine routed-path dependency based on downstream reporting preference

D MUST rely on routed execution path (MC) for MRS inclusion, but MUST NOT independently determine reporting dependency on MRS outputs.

If MRS is not part of the routed execution path:

- D MUST permit direct progression from Z to DB following lawful sequencing constraints.
- DB handoff is the final downstream handoff prior to `REPORT_READY`

D MUST NOT permit:

- DB handoff before execution outcome exists
- MRS handoff before structural outputs are available
- parallel downstream handoff branches for the same artifact path
- handoff reordering caused by user prompts, commands, or visibility requests

If lawful sequencing cannot be preserved, D MUST:

- block progression
- emit HANDOFF_VIOLATION

D emits failure_class only.

D MUST NOT:

- construct user-visible failure output
- define failure structure
- produce reporting content

Failure expression is exclusively owned by DB.

STRUCTURAL FLOW NORMALIZATION (LOCKED)

D governs structural readiness, wait-state normalization, and scope continuity only.

CONTAINER WAIT STATE

If containers are present but not fully processed:

- D MUST block progression beyond UNPACK_REQUIRED
- D MUST preserve the same artifact-path scope
- D MUST prohibit premature transition to ASSESS_READY or EXECUTE_READY

MULTI-ARTIFACT HANDLING

If multiple artifacts exist:

- D MUST ensure all artifacts in the active governed set are processed before progression
- D MUST preserve the governed set as a single progression scope unless artifact mechanics strictly require segmentation

No partial progression permitted.

No scope drift permitted.

If continuation is required for any artifact within the governed set:

- the full governed artifact set identity MUST be preserved without reduction, expansion, or reordering
- D MUST preserve the full set as continuation-bound
- D MUST NOT advance any subset independently

HANDOFF VIOLATION (LOCKED)

If D-owned conditions for stage transition, route continuity, continuation resumption, or lawful handoff sequencing are not met:

D MUST:

- block progression
- emit HANDOFF_VIOLATION

D MUST NOT:

- attempt recovery
- reroute execution
- create alternate workflow branches

- substitute user-direction for governed progression

NON-BRANCHING RULE (LOCKED)

D MUST NOT create alternate execution paths, alternate stage ladders, or semantically overlapping next-stage branches for the same governed artifact path.

Identify → Unpack → Assess → Execute → Report

All governed artifact paths MUST conform to the normalized stage progression model, with conditional stage inclusion and routed-path-dependent downstream completion behavior preserved.

Branching is permitted only where strictly required by artifact mechanics and only if canonical progression and governed handoff continuity are preserved.

SINGLE OWNER ENFORCEMENT (LOCKED)

D MUST NOT:

- perform classification
- enforce execution rules
- construct proof
- define reporting
- render output

Violation constitutes system defect.